Plectis: Making Claims About an AI-Built System Runnable

A first-principles account of the public repository

Will Cook

July 2026

Abstract

An AI-built software system can accumulate many claims: that a safety check ran, that a proof attempt used the stated premises, that a generated page matches its source, or that interrupted work can be resumed. A list of such claims is difficult to assess because the reader must discover what ran, on which input, where the result was recorded, and where the claim stops. Plectis turns each selected claim into a small public object: a local command, a bounded input or replay, a result, an evidence handle, and an explicit limit. This paper explains the public repository through one complete run, then shows how its 88 components share the same runtime and evidence model. It also states the main limitation: Plectis is an executable cross-section, not a copy of a private system and not an authority to release software, change source, call providers, certify security, or declare a theorem proved.

1 The object in one sentence

Plectis is a public collection of runnable checks for selected claims made by an AI-built workflow and research system.

"Runnable" is the important word. A reader does not have to accept a prose description of a mechanism. Each component names a command or replay, the small input it acts on, the result it writes, the evidence behind that result, and a sentence saying what the result does *not* establish. The source is available at github.com/wcook04/plectis. It is a local Python tool with no third-party runtime dependencies; the normal first command is:

```
plectis tour --format text <project>
```

This definition is deliberately narrower than "an AI operating system" or "an autonomous research laboratory." Those phrases describe ambitions too broad to inspect in one public repository. Plectis publishes bounded pieces that an outsider can run and challenge.

2 Why a normal description is not enough

Suppose a project page says, "the system preserves evidence." Four questions immediately follow.

1. What operation produced the evidence?
2. What exact files or events support it?
3. Can somebody else repeat the operation?
4. Does the result justify only a local observation, or a larger claim about safety, correctness, or release readiness?

A screenshot answers little. A test name is not enough either: a test can be green while exercising a toy input, omitting a dependency, or checking a weaker property than the prose suggests. Plectis therefore treats a public claim as a five-part object:

$$\boxed{\text{claim} + \text{command} + \text{input} + \text{receipt} + \text{limit}}$$

The five parts do not make a claim true by themselves. They make it possible to locate the disagreement. A sceptic can challenge the input, the code, the recorded result, or the inference from the result to the public wording.

3 One run from beginning to end

On 17 July 2026, the source-form command below was run against the Plectis checkout itself:

```
PYTHONPATH=src python3 -m microcosm_core tour --format text .
```

The command reported that it read 5,041 project files, selected the `readme_onboarding_route` from five candidates, and wrote 68 local record files under `.microcosm/`. It also reported three handles: `.microcosm/evidence/` for supporting artifacts, `.microcosm/events.jsonl` for the causal record, and a scope statement that grants no release, provider-call, or whole-system authority. File counts will change with the checkout; the important result is the shape of the run.

Input	A folder chosen by the user.
Action	Index the folder, discover patterns, select a route, and record why.
Output	Project-local state under `.microcosm/` and a plain-text card.
Evidence	Events and receipt files that can be reopened separately.
Limit	No source mutation, provider call, release decision, or correctness proof.

The source folder remains the user's source. Plectis writes beside it in a git-ignored directory. The record contains a catalog, discovered patterns, route candidates, work records, events, explanations, and evidence. A second command can return the same first screen as machine-readable data:

```
plectis tour --card <project>
```

This example is small enough to audit. The reader can inspect the selected route, follow its evidence handles, remove the local record, rerun the command, and compare the result. Nothing in this run claims that the chosen route is the best possible route or that the inspected project is correct. It proves a smaller fact: the public runtime can turn this folder into this bounded, traceable local record without changing the source.

4 The machinery needed to explain that run

The runtime uses a shared sequence of ordinary operations. It names the project, catalogs recognisable file roles, records patterns, proposes routes, creates reversible work records, emits

events, keeps evidence, and explains how a route was chosen. In the source these shared operations are called *kernel primitives*. The name is less important than the constraint: components bind to the same small runtime instead of inventing a separate framework for every check.

Part	Job
Command surface	Receives every `plectis` command.
Runtime spine	Turns a folder into catalog, route, work, event, and evidence records.
Components	Supply one bounded mechanism or replay each.
Validators	Check the result shape and evidence expected from a component.
Governed registries	Hold the machine-readable component and evidence contracts.
Projections	Build public maps and pages from those registries.

The public architecture page is generated from the registries. This matters because a hand-maintained list can silently describe components that have moved or disappeared. The generated page is still only a view; the registry, source modules, fixtures, and receipts remain the stronger evidence.

5 From one run to 88 components

The repository contains 88 accepted component contracts in seven families. The count describes the inventory, not maturity or quality.

Family	Count	Examples of its job
Entry and reveal	2	Give a cold reader a first route and public walkthrough.
Architecture and navigation	12	Discover patterns, route questions, and expose standards.
Formal mathematics and proof	20	Exercise proof-adjacent evidence, premise, tactic, and certificate flows.
Agent reliability and safety replays	20	Replay bounded failures involving prompts, tools, memory, sandboxes, and evaluation.
Research and science replays	9	Exercise replication, forecasting, world-model, and lab-safety workflow shapes.
Import, projection, and drift	20	Track source imports and detect stale generated views.
Work, landing, and continuity	5	Record reversible work, landing decisions, and resumable runs.

A component is not a plugin with unrestricted access. It is closer to an executable specimen. For example, a safety replay can show how one frozen prompt-injection case is detected by one stated rule. It cannot certify a production agent against every attack. A formal-mathematics component can show that a proof-evidence route produced the required witness shape. It cannot replace the Lean kernel or declare an open theorem solved.

The same five questions apply to every family: what is claimed, what runs, what input is used, what receipt is produced, and where authority stops. This shared reading rule is the reason a large, varied corpus can remain inspectable.

6 Evidence is typed, not merely attached

Not all receipts deserve equal weight. A row copied from source is different from a subprocess execution; a synthetic fixture is different from a live provider run; a generated index is different from proof authority. Plectis records an *evidence class* so the reader knows what kind of support is being offered.

The practical discipline is:

1. the public claim names its evidence class;
2. the component names the source or runner that produces it;
3. a validator checks the expected output and boundary fields;
4. a receipt records the actual result; and
5. the authority ceiling prevents a local pass from being promoted into a stronger claim.

This is not a universal truth machine. A dishonest input can still yield an honestly recorded result about the wrong case. A validator can faithfully check an incomplete contract. Human review is still required to decide whether the public sentence is the right sentence and whether the example is representative. The machinery makes those review points visible.

7 The public/private boundary

Plectis originated as a public cross-section of a larger private workflow. That relationship is provenance, not equivalence. The public repository does not expose private state, depend on private files at runtime, or claim to reconstruct the whole private system. A component may preserve a non-secret mechanism, a fixture, or a replay shape while omitting private data and live authority.

Four boundaries are especially important:

- **No provider authority.** A local result does not authorize a model, cloud, or external service call.
- **No mutation authority.** The inspection runtime does not gain permission to change the user's source.
- **No release authority.** Passing a component or test suite does not decide that software should be published.
- **No domain authority.** Replays concerning security, finance, science, or formal proof are bounded demonstrations, not professional approval or correctness oracles.

These are stored as fields and checked by tests because a disclaimer at the end of a page is easy to lose. A component that cannot state its boundary is not ready to support a public claim.

8 How a sceptical reader can inspect it

The shortest useful review is not to read every component. It is to take one claim through the whole chain.

1. Run `plectis tour -format text .` and check that the described local record exists.
2. Ask `plectis comprehend -first-action "<claim>" -format text` to locate the owning component, test command, and authority ceiling.
3. Open the component card in `ORGANS.md`; follow its runner, source, evidence-class, and receipt links.

4. Run the named fixture or validator. Compare the new result with the tracked receipt rather than trusting the prose summary.
5. Read the non-claim. Decide whether the public wording stays within it.

For a wider verification pass, `make check` loads the evidence registry and proof-trust floor. `make ci` adds the public tests, smoke path, and fresh package-install check. Those commands establish that the checked public contracts run in the current checkout. They do not establish that every important claim has been selected or that every selected contract is strong enough.

9 What this design contributes—and what it does not

The reusable idea is simple: publish claims as executable, bounded objects instead of asking a reader to trust a system overview. The shared runtime makes heterogeneous examples navigable; typed evidence prevents unlike forms of support from being treated as interchangeable; receipts make runs reopenable; authority ceilings keep local success local.

The present repository is still one project maintained under one direction. It does not contain an independent user study, a comparative evaluation against other workflow systems, or evidence that 88 is the right number of components. Many components use frozen or synthetic cases because that is what can be published and replayed safely. Such cases are useful for checking a mechanism's shape, but weak evidence for performance in an uncontrolled environment.

The central test is therefore not "How large is Plectis?" It is: can a reader choose a consequential claim, run the named mechanism, inspect the evidence, and see exactly where the inference ends? When the answer is no, the public object is incomplete even if its code passes.

10 Conclusion

Plectis makes selected parts of an AI-built system available in the smallest form that can be challenged: claim, command, input, receipt, and limit. One local tour shows the whole pattern. It reads a user-chosen folder, records a route and its evidence beside the folder, leaves the source unchanged, and states that the result grants no wider authority. The 88-component corpus repeats that contract across proof, safety, research, projection, and work continuity examples.

The repository should be judged by those inspectable chains, not by its vocabulary or its component count. Its public promise is not that the larger system is correct. It is that the reader can locate what was run, what it showed, what supports it, and what it does not permit them to conclude.